Guia de Sobrevivência Definitivo: Full-Stack Node.js + React

Manual de consulta rápida para avaliação prática presencial, individual e offline.

> **Convenções do guia:** Node 18+ (ideal 20+), módulos ESM (`import` / `export`), API em `http://localhost:3000` , front em `http://localhost:5173` , MongoDB em `mongodb://127.0.0.1:27017` . Todo o código é completo e pode ser copiado como está.

Mapa final de pastas

```
prova-fullstack/
├── backend/
│   ├── .env
│   ├── package.json
│   └── src/
│       ├── server.js
│       ├── config/db.js
│       ├── models/ (User.js, Task.js)
│       ├── controllers/ (authController.js, taskController.js)
│       ├── middlewares/authMiddleware.js
│       ├── routes/ (authRoutes.js, taskRoutes.js, reportRoutes.js)
│       └── sockets/socket.js
└── frontend/
    ├── .env, index.html, vite.config.js, package.json
    └── src/
        ├── main.jsx, index.css, App.jsx
        ├── services/api.js
        ├── components/ (PrivateRoute.jsx, GraficoTarefas.jsx,
TarefasTempoReal.jsx)
        └── pages/ (Auth.jsx, Dashboard.jsx)
```

SEÇÃO 1: SETUP E CONFIGURAÇÃO INICIAL (CLI & PACKAGE.JSON)

1.1 Pré-checagem do ambiente

```
# versão do Node (precisa ser 18 ou maior)
node -v
# versão do npm
npm -v
# testa se o MongoDB está respondendo (deve imprimir ok: 1)
mongosh --eval 'db.runCommand({ ping: 1 })'
```

Se o MongoDB não estiver rodando: Linux `sudo systemctl start mongod` · Windows (PowerShell como administrador) `net start MongoDB` · macOS `brew services start mongodb-community`.

1.2 Criando as pastas pelo terminal

No Windows, use o **Git Bash** (o `mkdir -p` e o `touch` funcionam como no Linux).

```
# cria a pasta raiz do projeto e entra nela
mkdir prova-fullstack
cd prova-fullstack

# ---------- BACKEND ----------
# cria a pasta do backend e entra nela
mkdir backend
cd backend
# cria todas as subpastas de uma vez (-p cria também as pastas
intermediárias)
mkdir -p src/config src/models src/controllers src/middlewares src/routes
src/sockets
# cria os arquivos vazios que serão preenchidos nas próximas seções
touch .env .gitignore src/server.js src/config/db.js src/models/User.js
src/models/Task.js src/controllers/authController.js
src/controllers/taskController.js src/middlewares/authMiddleware.js
src/routes/authRoutes.js src/routes/taskRoutes.js src/routes/reportRoutes.js
src/sockets/socket.js
# volta para a raiz do projeto
cd ..

# ---------- FRONTEND ----------
# cria a pasta do frontend e entra nela
mkdir frontend
cd frontend
```

```
# cria as subpastas do código-fonte
mkdir -p src/services src/pages src/components
# cria os arquivos vazios do frontend
touch .env .gitignore index.html vite.config.js src/main.jsx src/index.css
src/App.jsx src/services/api.js src/components/PrivateRoute.jsx
src/components/GraficoTarefas.jsx src/components/TarefasTempoReal.jsx
src/pages/Auth.jsx src/pages/Dashboard.jsx
# volta para a raiz do projeto
cd ..
```

Conteúdo do `.gitignore` (nos dois projetos):

```
node_modules
.env
dist
```

> **Alternativa oficial para o front:** `npm create vite@latest frontend -- --template react`. Se o assistente perguntar sobre *rolldown* ou *install now*, responda **No**. Depois substitua o `package.json` gerado pelo da seção 1.4. A criação manual acima é mais previsível porque não depende de perguntas interativas.

1.3 `package.json` COMPLETO do Backend

Salve em `backend/package.json`.

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "API REST da prova prática (Express + Mongoose)",
  "main": "src/server.js",
  "type": "module",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "engines": {
    "node": ">=18"
  },
  "dependencies": {
    "argon2": "^0.41.1",
    "cookie-parser": "^1.4.7",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
```

```
    "mongoose": "^8.9.5",
    "pdfkit": "^0.16.0",
    "socket.io": "^4.8.1",
    "swagger-ui-express": "^5.0.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.9"
  }
}
```

Pontos críticos do arquivo: `"type": "module"` habilita `import/export` (sem ele, só `require`). `npm start` roda em produção com `node`; `npm run dev` usa o `nodemon`, que reinicia o servidor a cada arquivo salvo.

Biblioteca	Papel no projeto	Uso típico
`express`	Framework HTTP: rotas, middlewares, req/res	`app.get('/x', handler)`
`mongoose`	ODM do MongoDB: schemas, validação, consultas	`Model.find()`
`argon2`	Hash seguro de senha (Argon2id)	`argon2.hash(senha)` e `argon2.verify(hash, senha)`
`jsonwebtoken`	Cria e valida tokens JWT	`jwt.sign(...)` e `jwt.verify(...)`
`cookie-parser`	Preenche `req.cookies` a partir do header Cookie	`req.cookies.token`
`socket.io`	WebSocket com eventos, salas e reconexão	`io.emit('evento', dados)`
`pdfkit`	Gera PDF por código, em stream	`new PDFDocument()`
`swagger-ui-express`	Serve a documentação interativa da API	`app.use('/api-docs', ...)`
`cors`	Libera a origem do front para chamar a API	`cors({ origin, credentials: true })`
`dotenv`	Carrega o arquivo `.env` em `process.env`	`process.env.MONGO_URI`
`nodemon` (dev)	Reinicia o servidor ao salvar arquivos	`npm run dev`

1.4 `package.json` COMPLETO do Frontend

Salve em `frontend/package.json`.

```
{
  "name": "frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "apexcharts": "^4.3.0",
    "axios": "^1.7.9",
    "react": "^19.0.0",
    "react-apexcharts": "^1.7.0",
    "react-dom": "^19.0.0",
    "react-router-dom": "^7.1.1",
    "socket.io-client": "^4.8.1"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "tailwindcss": "^4.0.0",
    "vite": "^6.0.5"
  }
}
```

Biblioteca	Papel no projeto	Uso típico
`react` / `react-dom`	Biblioteca de UI e renderização no navegador	`createRoot(...).render(<App />)`
`vite`	Servidor de desenvolvimento e bundler rápido	`npm run dev`
`@vitejs/plugin-react`	Plugin do Vite que compila JSX	`plugins: [react()]`
`tailwindcss`	Framework de classes utilitárias (v4)	`className='flex gap-4'`

Biblioteca	Papel no projeto	Uso típico
`@tailwindcss/vite`	Plugin oficial do Tailwind v4 para o Vite (substitui PostCSS e `tailwind.config.js`)	`plugins: [tailwindcss()]`
`react-router-dom`	Rotas no cliente, navegação sem recarregar	`<Routes>`, `<Route>`, `useNavigate`
`axios`	Cliente HTTP com instância, cookies e interceptors	`api.get('/tasks')`
`socket.io-client`	Cliente WebSocket compatível com o servidor	`io(URL, { withCredentials: true })`
`apexcharts`	Motor dos gráficos	dependência obrigatória do wrapper
`react-apexcharts`	Componente React para o ApexCharts	`<Chart options series type />`

1.5 Comando mágico para a hora da prova

Antes da prova (com internet): crie a pasta `~/kit-prova` com `backend/package.json` e `frontend/package.json` (os dois arquivos acima). Rode `npm i` uma vez em cada projeto na sua máquina para popular o cache do npm e gerar o `package-lock.json`; guarde também os dois `package-lock.json` no kit.

Na prova, backend:

```
cd ~/prova-fullstack/backend && cp ~/kit-prova/backend/package.json . && npm i --prefer-offline && npm run dev
```

Na prova, frontend (outro terminal):

```
cd ~/prova-fullstack/frontend && cp ~/kit-prova/frontend/package.json . && npm i --prefer-offline && npm run dev
```

Script único `setup.sh` (guarde no kit; roda com `bash ~/kit-prova/setup.sh`):

```
#!/usr/bin/env bash
# interrompe o script no primeiro comando que falhar
set -e
# pasta do kit com os package.json prontos
KIT=$HOME/kit-prova
# pasta do projeto que será criada
ROOT=$HOME/prova-fullstack
```

```
# cria a estrutura de pastas do backend e do frontend
mkdir -p $ROOT/backend/src/config $ROOT/backend/src/models
$ROOT/backend/src/controllers $ROOT/backend/src/middlewares
$ROOT/backend/src/routes $ROOT/backend/src/sockets
mkdir -p $ROOT/frontend/src/services $ROOT/frontend/src/pages
$ROOT/frontend/src/components

# copia os package.json (e os locks, se existirem) do kit para o projeto
cp $KIT/backend/package.json $ROOT/backend/
cp $KIT/frontend/package.json $ROOT/frontend/
cp -f $KIT/backend/package-lock.json $ROOT/backend/ 2>/dev/null || true
cp -f $KIT/frontend/package-lock.json $ROOT/frontend/ 2>/dev/null || true

# instala as dependências usando o cache local sempre que possível
cd $ROOT/backend && npm i --prefer-offline
cd $ROOT/frontend && npm i --prefer-offline

# mensagem final
echo 'Pronto: backend com npm run dev, frontend com npm run dev'
```

> **Sem internet na prova?** O `npm i` precisa do registro. Três saídas, da melhor para a pior: (1) a máquina da prova tem espelho/proxy do npm configurado; (2) o cache `~/.npm` já foi populado na MESMA máquina e `--prefer-offline` ou `--offline` resolve; (3) você leva um `node_modules` compactado (o `argon2` é binário nativo e só funciona no mesmo SO, arquitetura e versão do Node).

SEÇÃO 2: DICIONÁRIO TÉCNICO E GUIA DE REFERÊNCIA RÁPIDA

2.1 Dicionário Mongoose (MongoDB)

Tipos de dados aceitos no Schema

Tipo	Sintaxe	Observações
String	`String`	aceita `trim`, `lowercase`, `uppercase`, `minlength`, `maxlength`, `match`, `enum`
Number	`Number`	aceita `min` e `max`

Tipo	Sintaxe	Observações
Date	`Date`	use `default: Date.now` **sem parênteses** (com parênteses fixa a data do boot)
Boolean	`Boolean`	`true` ou `false`
ObjectId	`Schema.Types.ObjectId`	referência a outro documento; sempre acompanhada de `ref`
Array	`[String]`, `[Number]`, `[{ type: Schema.Types.ObjectId, ref: 'User' }]`	lista de primitivos, de referências ou de subdocumentos
Subdocumento	`{ rua: String, numero: Number }`	objeto aninhado dentro do documento
Mixed	`Schema.Types.Mixed`	aceita qualquer estrutura, sem validação

Modificadores e propriedades úteis

Modificador	Função	Exemplo
`required`	Campo obrigatório; aceita `[true, 'mensagem']`	`required: [true, 'Nome obrigatório']`
`unique`	Cria índice único. **Não é validador**: o erro vem do Mongo com `code: 11000`	`unique: true`
`default`	Valor usado quando o campo não é enviado (valor ou função)	`default: 'pendente'`
`select: false`	Oculta o campo nas consultas. Para ler: `.select('+senha')`	`select: false`
`enum`	Restringe a uma lista de valores	`enum: ['pendente', 'feita']`
`min` / `max`	Limites de Number ou Date	`min: 0, max: 10`
`minlength` / `maxlength`	Limites de tamanho de String	`minlength: 6`
`match`	Valida a String por expressão regular	`match: [/^[0-9]{11}$/, 'CPF inválido']`

Modificador	Função	Exemplo
`validate`	Validador customizado (função que retorna boolean)	`validate: { validator: fn, message: 'texto' }`
`ref`	Nome do Model referenciado, usado pelo `populate`	`ref: 'User'`
`trim` / `lowercase`	Normalizam a String antes de salvar	`trim: true`
`immutable`	Impede alteração depois de criado	`immutable: true`

Schema de referência usando TODOS os modificadores

```
// importa o mongoose (default export do pacote)
import mongoose from 'mongoose';
// extrai o construtor Schema para escrever menos código
const { Schema } = mongoose;

// cria o schema: primeiro argumento = campos, segundo = opções do schema
const exemploSchema = new Schema(
  {
    // String obrigatória, sem espaços nas pontas, entre 3 e 80 caracteres
    nome: { type: String, required: [true, 'O nome é obrigatório'], trim:
true, minlength: [3, 'Mínimo de 3 caracteres'], maxlength: 80 },
    // e-mail único, sempre minúsculo, validado por regex
    email: { type: String, required: true, unique: true, lowercase: true,
match: [/^[a-z0-9._-]+@[a-z0-9.-]+[.][a-z]{2,}$/i, 'E-mail inválido'] },
    // senha oculta nas consultas (precisa de .select('+senha') para ler)
    senha: { type: String, required: true, minlength: 6, select: false },
    // número com limites e valor padrão
    idade: { type: Number, min: [0, 'Idade mínima é 0'], max: [120, 'Idade
máxima é 120'], default: 18 },
    // string restrita a uma lista, com mensagem customizada ({VALUE} é o
valor recebido)
    papel: { type: String, enum: { values: ['aluno', 'professor', 'admin'],
message: '{VALUE} não é um papel válido' }, default: 'aluno' },
    // boolean com padrão
    ativo: { type: Boolean, default: true },
    // data com padrão = momento da criação (Date.now SEM parênteses)
    nascimento: { type: Date, default: Date.now },
    // array de strings que começa vazio
    tags: { type: [String], default: [] },
    // subdocumento (objeto aninhado)
    endereco: { rua: String, numero: Number },
```

```
    // validação customizada: exatamente 11 dígitos
    cpf: { type: String, validate: { validator: (valor) => /^[0-9]
{11}$/.test(valor), message: 'CPF deve ter 11 dígitos' } },
    // referência 1:1 a um usuário (guarda apenas o _id)
    dono: { type: Schema.Types.ObjectId, ref: 'User', required: true,
immutable: true },
    // array de referências 1:N
    amigos: [{ type: Schema.Types.ObjectId, ref: 'User' }],
  },
  // timestamps: true cria e mantém createdAt e updatedAt automaticamente
  { timestamps: true }
);

// registra o model: 'Exemplo' vira a collection 'exemplos' no MongoDB
export default mongoose.model('Exemplo', exemploSchema);
```

Métodos essenciais no Controller

```
// lista todos os documentos que casam com o filtro (retorna array, vazio se
nada casar)
const lista = await Task.find({ status: 'pendente' }).sort({ createdAt: -1
}).limit(10).select('titulo status');
// primeiro documento que casa com o filtro (ou null)
const usuario = await User.findOne({ email: 'a@a.com' });
// busca por _id (ou null); lança CastError se o id tiver formato inválido
const tarefa = await Task.findById(id);
// cria e salva um documento novo (roda as validações do schema)
const nova = await Task.create({ titulo: 'Estudar', owner: userId });
// busca, atualiza e devolve; new: true devolve o documento JÁ atualizado;
runValidators aplica o schema
const atualizada = await Task.findOneAndUpdate({ _id: id, owner: userId }, {
titulo: 'Novo' }, { new: true, runValidators: true });
// busca por _id e remove (devolve o documento removido ou null)
const removida = await Task.findByIdAndDelete(id);
// troca o _id guardado em owner pelo documento completo (só os campos nome e
email)
const comDono = await Task.findById(id).populate('owner', 'nome email');
// conta documentos
const total = await Task.countDocuments({ owner: userId });
// devolve true/false sem trazer o documento
const existe = await User.exists({ email: 'a@a.com' });
// agrupa e soma (usado nos gráficos)
const grupos = await Task.aggregate([{ $group: { _id: '$status', total: {
$sum: 1 } } }]);
```

Operadores de filtro: `{ idade: { $gt: 18 } }` maior · `$gte` maior ou igual · `$lt` menor · `$in: ['a','b']` dentro da lista · `$ne` diferente · `$regex: 'ana', $options: 'i'` busca textual · `$or: [{ a: 1 }, { b: 2 }]` ou.

Mapa de erros do Mongoose para HTTP:

Situação	Como detectar	Status
Falha de validação do schema	`err.name === 'ValidationError'`	400
`id` com formato inválido	`err.name === 'CastError'`	400
Valor duplicado em campo `unique`	`err.code === 11000`	400
Documento não encontrado	resultado `null`	404
Qualquer outro erro	`catch` genérico	500

2.2 Dicionário React (Hooks e Navegação)

`useState`: memória do componente

```
// importa o hook
import { useState } from 'react';

// [valorAtual, funçãoQueAltera] = useState(valorInicial); alterar o valor
re-renderiza o componente
const [contador, setContador] = useState(0);
// forma funcional: use quando o novo valor depende do anterior
setContador((anterior) => anterior + 1);
// estado de formulário como objeto: um único estado para todos os campos
const [form, setForm] = useState({ email: '', senha: '' });
// atualização imutável: o operador spread (três pontos) copia o objeto e
[name] sobrescreve só o campo digitado
const handleChange = (e) => setForm({ ...form, [e.target.name]:
e.target.value });
// NUNCA faça form.email = 'x' direto: o React não percebe a mudança
```

`useEffect`: efeitos colaterais e ciclo de vida

```
import { useEffect, useState } from 'react';

function Exemplo({ id }) {
  const [dados, setDados] = useState([]);

  // 1) SEM array: roda após TODA renderização (quase nunca desejado)
```

```
  useEffect(() => { console.log('renderizou'); });

  // 2) array VAZIO []: roda UMA vez, na montagem do componente (carga
inicial de dados)
  useEffect(() => {
    // o callback do useEffect não pode ser async; declare uma função async
dentro e chame
    const carregar = async () => {
      const resposta = await api.get('/tasks');
      setDados(resposta.data);
    };
    carregar();
  }, []);

  // 3) array com DEPENDÊNCIAS [id]: roda na montagem e de novo sempre que id
mudar
  useEffect(() => { console.log('id mudou para', id); }, [id]);

  // 4) CLEANUP: a função retornada roda na DESMONTAGEM (ou antes da próxima
execução)
  useEffect(() => {
    const intervalo = setInterval(() => console.log('tick'), 1000);
    // sem esta limpeza o intervalo (ou socket, ou listener) continua vivo e
vaza memória
    return () => clearInterval(intervalo);
  }, []);

  return null;
}
```

Ciclo de vida	Como fazer com `useEffect`
Montou (componentDidMount)	`useEffect(fn, [])`
Atualizou algo específico	`useEffect(fn, [dependencia])`
Vai desmontar (componentWillUnmount)	`return () => limpeza` dentro do efeito

> Em desenvolvimento, o `<StrictMode>` executa montagem, limpeza e montagem de novo de propósito. Por isso a limpeza é obrigatória em sockets e timers.

`useNavigate`, `useParams` e `Link`

```
import { Link, useNavigate, useParams } from 'react-router-dom';

// rota declarada em App.jsx: <Route path='/tarefas/:id' element={<Detalhe
/>} />
function Detalhe() {
  // useParams devolve um objeto com os parâmetros da URL; aqui { id: '123' }
(sempre String)
  const { id } = useParams();
  // useNavigate devolve a função de redirecionar por código
  const navigate = useNavigate();

  // navigate('/rota') troca a página; { replace: true } não deixa a página
atual no histórico; navigate(-1) volta
  const voltar = () => navigate('/dashboard', { replace: true });

  return (
    <div>
      <p>Tarefa {id}</p>
      <button onClick={voltar}>Voltar</button>
      {/* Link navega sem recarregar a página (use no lugar de <a href>) */}
      <Link to='/dashboard'>Ir para o painel</Link>
    </div>
  );
}
```

Padrão de consumo de API com Axios

```
// api é a instância base (arquivo completo na Seção 4.2) com baseURL e
withCredentials: true
import api from '../services/api';

// GET: dados chegam em resposta.data
const resposta = await api.get('/tasks');
// POST: segundo argumento é o body JSON
await api.post('/tasks', { titulo: 'Estudar' });
// PUT: id na URL, body com os campos alterados
await api.put('/tasks/' + id, { status: 'feita' });
// DELETE: só a URL
await api.delete('/tasks/' + id);

// padrão de tratamento de erro: o Axios lança erro para status fora de 2xx
try {
  await api.post('/auth/login', form);
} catch (err) {
```

```
  // err.response existe quando o servidor respondeu (400, 401, 500);
  // err.response.data.message é a mensagem que o backend enviou; sem
response = servidor fora do ar
  const mensagem = err.response?.data?.message || 'Servidor indisponível';
  console.log(err.response?.status, mensagem);
}
```

2.3 Cheatsheet Tailwind CSS v4

Setup v4: o CSS tem uma única linha, `@import 'tailwindcss';`, e o plugin `@tailwindcss/vite` entra no `vite.config.js`. Não existe `tailwind.config.js`. Personalização via `@theme { --color-marca: #4f46e5; }` gera `bg-marca`, `text-marca`.

Mudanças da v4 que pegam: `bg-gradient-to-r` virou `bg-linear-to-r` · `shadow-sm` virou `shadow-xs` e `shadow` virou `shadow-sm` · `rounded-sm` virou `rounded-xs` · `outline-none` virou `outline-hidden` · `flex-shrink-0` virou `shrink-0` · `ring` agora tem 1px (use `ring-2` ou `ring-3`).

Categoria	Classes essenciais	O que fazem
Flexbox	`flex` `inline-flex` `flex-col` `flex-row` `flex-wrap` `flex-1` `shrink-0`	Ativa flex, define direção, quebra de linha, ocupa espaço sobrando
	`items-center` `items-start` `items-end`	Alinhamento no eixo cruzado
	`justify-center` `justify-between` `justify-around` `justify-end`	Distribuição no eixo principal
	`gap-4` `gap-x-2` `gap-y-6` `space-y-4`	Espaço entre filhos
Grid	`grid` `grid-cols-1` `grid-cols-3` `md:grid-cols-3` `col-span-2` `place-items-center`	Grade responsiva e posicionamento
Espaçamento	`p-4` `px-4` `py-2` `pt-6` `m-4` `mx-auto` `mt-6` `mb-2` (1 unidade = 0.25rem)	Padding e margin; `mx-auto` centraliza bloco com largura
Dimensões	`w-full` `w-1/2` `w-64` `h-10` `h-screen` `min-h-screen` `max-w-md` `max-w-7xl` `size-10` `w-[300px]`	Largura, altura, limites; colchetes = valor arbitrário

Categoria	Classes essenciais	O que fazem
Tipografia	`text-sm` `text-xl` `text-3xl` `font-semibold` `font-bold` `text-center` `uppercase` `tracking-wide` `truncate`	Tamanho, peso, alinhamento, corte com reticências visuais
Cores	`text-gray-700` `text-white` `text-red-600` `bg-white` `bg-gray-100` `bg-blue-600` `bg-red-50` `bg-blue-600/80`	Texto e fundo; `/80` = 80% de opacidade
Bordas e sombras	`border` `border-gray-300` `rounded` `rounded-lg` `rounded-full` `shadow-md` `ring-2`	Contorno, cantos, sombra
Estados	`hover:bg-blue-700` `focus:outline-hidden` `focus:ring-2` `focus:ring-blue-500` `disabled:opacity-50` `disabled:cursor-not-allowed` `active:scale-95` `placeholder:text-gray-400` `transition`	Variantes aplicadas só no estado indicado
Responsivo	`sm:` `md:` `lg:` `xl:` (mobile-first)	`md:flex-row` = linha a partir de 768px

Receitas prontas (copie o `className`):

Elemento	className
Página centralizada	`flex min-h-screen items-center justify-center bg-gray-100`
Card	`w-full max-w-md rounded-xl bg-white p-8 shadow-md`
Input	`w-full rounded-lg border border-gray-300 px-3 py-2 focus:border-blue-500 focus:ring-2 focus:ring-blue-500 focus:outline-hidden`
Botão primário	`w-full rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white transition hover:bg-blue-700 disabled:cursor-not-allowed disabled:opacity-50`

Elemento	className
Botão perigo	`rounded-lg bg-red-600 px-3 py-1 text-sm font-semibold text-white hover:bg-red-700`
Alerta de erro	`rounded-lg border border-red-300 bg-red-50 px-4 py-2 text-sm text-red-700`
Tabela	`w-full text-left text-sm` com `<th className='border-b p-2 font-semibold'>` e `<td className='border-b p-2'>`

SEÇÃO 3: BOILERPLATE BACKEND (ARQUITETURA COMPLETA)

Fluxo de uma requisição: **rota → middleware de autenticação → controller → model → MongoDB → resposta JSON**. Cada pasta tem uma responsabilidade; na prova, crie os arquivos nesta ordem: `.env`, `db.js`, models, middleware, controllers, routes, `socket.js` (Seção 5.1) e por último `server.js`.

3.1 Variáveis de ambiente (`backend/.env`)

```
# porta em que a API escuta
PORT=3000
# endereço do MongoDB local; o final (/prova_db) é o nome do banco, criado automaticamente no primeiro insert
MONGO_URI=mongodb://127.0.0.1:27017/prova_db
# segredo usado para ASSINAR e VERIFICAR os tokens JWT (nunca vai para o Git)
JWT_SECRET=troque_esta_chave_por_uma_frase_longa_e_aleatoria
# validade do token (1d = 1 dia, 2h = 2 horas)
JWT_EXPIRES_IN=1d
# origem exata do frontend, usada no CORS e no Socket.io
CLIENT_URL=http://localhost:5173
# development ou production (em production o cookie exige HTTPS)
NODE_ENV=development
```

3.2 Conexão com o banco (`src/config/db.js`)

```
// importa o mongoose, responsável por conectar e modelar o MongoDB
import mongoose from 'mongoose';

// exporta a função de conexão; o server.js chama antes de abrir a porta
export default async function conectarBanco() {
```

```
  try {
    // lê a URI definida no .env
    const uri = process.env.MONGO_URI;
    // sem URI não há o que conectar: lança erro com mensagem clara
    if (!uri) {
      throw new Error('MONGO_URI não definida no arquivo .env');
    }
    // strictQuery true: campos fora do schema são ignorados nos filtros
    mongoose.set('strictQuery', true);
    // abre a conexão; falha em 5 segundos se o MongoDB estiver desligado (o
padrão seria 30s)
    const conexao = await mongoose.connect(uri, { serverSelectionTimeoutMS:
5000 });
    // confirma no terminal qual host e banco foram conectados
    console.log('MongoDB conectado em ' + conexao.connection.host + '/' +
conexao.connection.name);
  } catch (erro) {
    // mostra o motivo da falha
    console.error('Falha ao conectar no MongoDB:', erro.message);
    // encerra o processo com código 1: sem banco a API não deve subir
    process.exit(1);
  }
}

// aviso se a conexão cair depois de estabelecida
mongoose.connection.on('disconnected', () => console.warn('MongoDB
desconectado'));
```

3.3 Servidor principal (`src/server.js`)

```
// ATENÇÃO: este import precisa ser o PRIMEIRO; ele carrega o .env em
process.env antes de tudo
import 'dotenv/config';
// módulo http nativo do Node: necessário para Express e Socket.io dividirem
a mesma porta
import http from 'http';
// framework web
import express from 'express';
// middleware que libera o acesso do frontend (outra porta = outra origem)
import cors from 'cors';
// middleware que transforma o header Cookie em req.cookies
import cookieParser from 'cookie-parser';
// classe do servidor WebSocket
import { Server } from 'socket.io';
// serve a interface do Swagger
import swaggerUi from 'swagger-ui-express';
```

```
// função de conexão com o banco (3.2)
import conectarBanco from './config/db.js';
// função que registra os eventos do Socket.io (arquivo completo na Seção
5.1)
import registrarSockets from './sockets/socket.js';
// grupos de rotas da API
import authRoutes from './routes/authRoutes.js';
import taskRoutes from './routes/taskRoutes.js';
import reportRoutes from './routes/reportRoutes.js';

// cria a aplicação Express
const app = express();
// cria o servidor HTTP nativo e entrega o Express a ele como handler
const httpServer = http.createServer(app);

// origem do frontend; usa o .env e cai para o valor padrão do Vite
const CLIENT_URL = process.env.CLIENT_URL || 'http://localhost:5173';

// CORS com origem EXATA e credentials true: sem isso o navegador bloqueia o
envio de cookies
// (origin '*' é proibido quando credentials é true)
app.use(cors({ origin: CLIENT_URL, credentials: true }));
// lê body JSON e o coloca em req.body
app.use(express.json());
// lê cookies e os coloca em req.cookies
app.use(cookieParser());

// cria o servidor Socket.io acoplado ao httpServer, com o mesmo CORS da API
const io = new Server(httpServer, { cors: { origin: CLIENT_URL, credentials:
true } });
// guarda o io dentro do app para os controllers emitirem eventos via
req.app.get('io')
app.set('io', io);
// registra conexão, desconexão e eventos customizados
registrarSockets(io);

// documentação OpenAPI mínima, escrita como objeto JavaScript (não precisa
de YAML)
const swaggerDocument = {
  openapi: '3.0.0',
  info: { title: 'API da Prova', version: '1.0.0', description: 'CRUD de
tarefas com autenticação JWT em cookie' },
  servers: [{ url: 'http://localhost:3000/api' }],
  components: {
    // declara que a autenticação vem de um cookie chamado token
    securitySchemes: { cookieAuth: { type: 'apiKey', in: 'cookie', name:
```

```
'token' } },
  },
  paths: {
    '/auth/register': {
      post: {
        tags: ['Auth'],
        summary: 'Cadastra um usuário',
        requestBody: {
          required: true,
          content: { 'application/json': { schema: { type: 'object',
required: ['nome', 'email', 'senha'], properties: { nome: { type: 'string' },
email: { type: 'string' }, senha: { type: 'string' } } } } },
        },
        responses: { 201: { description: 'Usuário criado' }, 400: {
description: 'Dados inválidos ou e-mail já usado' } },
      },
    },
    '/auth/login': {
      post: {
        tags: ['Auth'],
        summary: 'Autentica e grava o cookie token',
        requestBody: {
          required: true,
          content: { 'application/json': { schema: { type: 'object',
required: ['email', 'senha'], properties: { email: { type: 'string' }, senha:
{ type: 'string' } } } } },
        },
        responses: { 200: { description: 'Login realizado' }, 401: {
description: 'Credenciais inválidas' } },
      },
    },
    '/tasks': {
      get: { tags: ['Tarefas'], summary: 'Lista as tarefas do usuário',
security: [{ cookieAuth: [] }], responses: { 200: { description: 'Lista de
tarefas' }, 401: { description: 'Não autenticado' } } },
      post: {
        tags: ['Tarefas'],
        summary: 'Cria uma tarefa',
        security: [{ cookieAuth: [] }],
        requestBody: { required: true, content: { 'application/json': {
schema: { type: 'object', required: ['titulo'], properties: { titulo: { type:
'string' }, descricao: { type: 'string' }, status: { type: 'string' } } } } }
},
        responses: { 201: { description: 'Tarefa criada' }, 400: {
description: 'Dados inválidos' } },
      },
```

```
    },
    '/tasks/{id}': {
      put: { tags: ['Tarefas'], summary: 'Atualiza uma tarefa', security: [{
cookieAuth: [] }], parameters: [{ name: 'id', in: 'path', required: true,
schema: { type: 'string' } }], responses: { 200: { description: 'Atualizada'
}, 404: { description: 'Não encontrada' } } },
      delete: { tags: ['Tarefas'], summary: 'Remove uma tarefa', security: [{
cookieAuth: [] }], parameters: [{ name: 'id', in: 'path', required: true,
schema: { type: 'string' } }], responses: { 200: { description: 'Removida' },
404: { description: 'Não encontrada' } } },
    },
  },
};

// disponibiliza a interface em http://localhost:3000/api-docs
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));

// rota de saúde: útil para testar se a API está no ar
app.get('/api/health', (req, res) => res.status(200).json({ status: 'ok' }));

// monta cada grupo de rotas sob seu prefixo
app.use('/api/auth', authRoutes);
app.use('/api/tasks', taskRoutes);
app.use('/api/reports', reportRoutes);

// middleware de 404: roda quando nenhuma rota acima respondeu
app.use((req, res) => res.status(404).json({ message: 'Rota não encontrada'
}));

// middleware de erro global: o Express o reconhece pelos QUATRO parâmetros
app.use((erro, req, res, next) => {
  console.error(erro);
  res.status(500).json({ message: 'Erro interno do servidor' });
});

// porta vinda do .env, com padrão 3000
const PORT = process.env.PORT || 3000;

// função que primeiro conecta no banco e só depois abre a porta
async function iniciar() {
  await conectarBanco();
  // IMPORTANTE: listen no httpServer, NÃO em app.listen (senão o Socket.io
não funciona)
  httpServer.listen(PORT, () => {
    console.log('API em http://localhost:' + PORT);
    console.log('Swagger em http://localhost:' + PORT + '/api-docs');
```

```
  });
}

// executa a inicialização
iniciar();
```

3.4 Models

src/models/User.js

```
// importa o mongoose
import mongoose from 'mongoose';

// define a estrutura e as regras da collection users
const userSchema = new mongoose.Schema(
  {
    // nome obrigatório, sem espaços nas pontas
    nome: { type: String, required: [true, 'O nome é obrigatório'], trim:
true, minlength: [2, 'O nome precisa de ao menos 2 caracteres'] },
    // e-mail obrigatório, único, sempre minúsculo e validado por regex
    email: {
      type: String,
      required: [true, 'O e-mail é obrigatório'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^[a-z0-9._-]+@[a-z0-9.-]+[.][a-z]{2,}$/i, 'E-mail inválido'],
    },
    // guarda o HASH do Argon2; select false esconde o campo em todas as
consultas
    // (o tamanho mínimo da senha é validado no controller, ANTES do hash,
pois o hash sempre é longo)
    senha: { type: String, required: [true, 'A senha é obrigatória'], select:
false },
    // papel do usuário, usado no middleware apenasAdmin
    papel: { type: String, enum: ['usuario', 'admin'], default: 'usuario' },
  },
  // cria createdAt e updatedAt automaticamente
  { timestamps: true }
);

// registra o model User (collection users)
export default mongoose.model('User', userSchema);
```

src/models/Task.js

```
// importa o mongoose
import mongoose from 'mongoose';

// atalho para o construtor de Schema
const { Schema } = mongoose;

// define a estrutura da collection tasks
const taskSchema = new Schema(
  {
    // título obrigatório com limite de tamanho
    titulo: { type: String, required: [true, 'O título é obrigatório'], trim:
true, maxlength: [100, 'Máximo de 100 caracteres'] },
    // descrição opcional, começa vazia
    descricao: { type: String, trim: true, default: '' },
    // status limitado a três valores; começa como pendente
    status: { type: String, enum: { values: ['pendente', 'andamento',
'concluida'], message: '{VALUE} não é um status válido' }, default:
'pendente' },
    // prioridade de 1 a 5
    prioridade: { type: Number, min: [1, 'Prioridade mínima é 1'], max: [5,
'Prioridade máxima é 5'], default: 3 },
    // prazo opcional
    prazo: { type: Date },
    // dono da tarefa: referência ao User; immutable impede trocar o dono
depois
    owner: { type: Schema.Types.ObjectId, ref: 'User', required: true,
immutable: true },
  },
  { timestamps: true }
);

// índice composto: acelera a listagem das tarefas de um usuário da mais nova
para a mais antiga
taskSchema.index({ owner: 1, createdAt: -1 });

// registra o model Task (collection tasks)
export default mongoose.model('Task', taskSchema);
```

3.5 Middleware de autenticação (`src/middlewares/authMiddleware.js`)

```
// biblioteca que valida a assinatura e a expiração do token
import jwt from 'jsonwebtoken';

// middleware: protege rotas exigindo um JWT válido (cookie ou header
Authorization)
```

```
export function protegerRota(req, res, next) {
  try {
    // 1ª fonte: cookie chamado token (preenchido pelo cookie-parser)
    let token = req.cookies?.token;
    // 2ª fonte: header 'Authorization: Bearer <token>' (útil em Postman e
Insomnia)
    const header = req.headers.authorization;
    if (!token && header && header.startsWith('Bearer ')) {
      // separa 'Bearer' do token e pega a segunda parte
      token = header.split(' ')[1];
    }
    // nenhuma fonte trouxe token: 401 = não autenticado
    if (!token) {
      return res.status(401).json({ message: 'Não autenticado: token ausente'
});
    }
    // verify lança erro se a assinatura for falsa ou o token estiver
expirado; devolve o payload se for válido
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    // anexa os dados do usuário à requisição para os próximos handlers
usarem
    req.userId = payload.id;
    req.userPapel = payload.papel;
    // libera a requisição para o próximo middleware ou controller
    next();
  } catch (erro) {
    // JsonWebTokenError e TokenExpiredError caem aqui
    return res.status(401).json({ message: 'Token inválido ou expirado' });
  }
}

// middleware: só deixa passar administradores (usar SEMPRE depois de
protegerRota)
export function apenasAdmin(req, res, next) {
  // 403 = autenticado, mas sem permissão
  if (req.userPapel !== 'admin') {
    return res.status(403).json({ message: 'Acesso negado: requer
administrador' });
  }
  // é admin: segue o fluxo
  next();
}
```

3.6 Controller de autenticação (`src/controllers/authController.js`)

```
// Argon2: hash e verificação de senha
import argon2 from 'argon2';
// JWT: criação do token
import jwt from 'jsonwebtoken';
// model de usuário
import User from '../models/User.js';

// configuração-base do cookie (usada ao gravar E ao limpar; precisam ser idênticas)
const baseCookie = {
  // httpOnly: o JavaScript do navegador NÃO consegue ler o cookie (protege contra XSS)
  httpOnly: true,
  // secure: em produção só trafega por HTTPS
  secure: process.env.NODE_ENV === 'production',
  // lax funciona em localhost (portas diferentes ainda são o mesmo site); em produção com domínios distintos usa none
  sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'lax',
};

// duração do cookie em milissegundos: 24h * 60min * 60s * 1000ms
const UM_DIA = 24 * 60 * 60 * 1000;

// gera o JWT com o id e o papel dentro do payload (nunca coloque senha no token)
function gerarToken(user) {
  return jwt.sign({ id: user._id, papel: user.papel },
process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || '1d',
  });
}

// POST /api/auth/register
export async function register(req, res) {
  try {
    // extrai os campos enviados no body
    const { nome, email, senha } = req.body;
    // validação de presença e de tipo: 400 = requisição malformada
    if (!nome || !email || typeof senha !== 'string') {
      return res.status(400).json({ message: 'Informe nome, e-mail e senha' });
    }
    // regra de negócio da senha (validada ANTES do hash)
    if (senha.length < 6) {
```

```
      return res.status(400).json({ message: 'A senha deve ter no mínimo 6
caracteres' });
    }
    // verifica duplicidade para devolver uma mensagem amigável
    const existe = await User.exists({ email: email.toLowerCase().trim() });
    if (existe) {
      return res.status(400).json({ message: 'E-mail já cadastrado' });
    }
    // gera o hash Argon2id (o salt é criado e embutido no próprio hash
automaticamente)
    const hash = await argon2.hash(senha);
    // grava o usuário com o HASH, nunca com a senha pura
    const user = await User.create({ nome, email, senha: hash });
    // 201 = recurso criado; devolve só dados públicos
    return res.status(201).json({
      message: 'Usuário cadastrado com sucesso',
      user: { id: user._id, nome: user.nome, email: user.email },
    });
  } catch (erro) {
    // erros de validação do schema viram 400 com a lista de mensagens
    if (erro.name === 'ValidationError') {
      const mensagens = Object.values(erro.errors).map((e) => e.message);
      return res.status(400).json({ message: mensagens.join(', ') });
    }
    // índice unique violado (condição de corrida): código 11000
    if (erro.code === 11000) {
      return res.status(400).json({ message: 'E-mail já cadastrado' });
    }
    console.error(erro);
    return res.status(500).json({ message: 'Erro interno ao cadastrar' });
  }
}

// POST /api/auth/login
export async function login(req, res) {
  try {
    const { email, senha } = req.body;
    if (!email || typeof senha !== 'string') {
      return res.status(400).json({ message: 'Informe e-mail e senha' });
    }
    // select('+senha') traz o campo que está oculto por select: false
    const user = await User.findOne({ email: email.toLowerCase().trim()
}).select('+senha');
    // mensagem idêntica para e-mail inexistente e senha errada (não revela
qual campo falhou)
    if (!user) {
```

```
      return res.status(401).json({ message: 'Credenciais inválidas' });
    }
    // verify(hashSalvo, senhaDigitada): ordem dos argumentos importa;
devolve true ou false
    const senhaCorreta = await argon2.verify(user.senha, senha);
    if (!senhaCorreta) {
      return res.status(401).json({ message: 'Credenciais inválidas' });
    }
    // gera o token e grava no cookie (httpOnly + validade de 1 dia)
    const token = gerarToken(user);
    res.cookie('token', token, Object.assign({}, baseCookie, { maxAge: UM_DIA
}));
    return res.status(200).json({
      message: 'Login realizado com sucesso',
      user: { id: user._id, nome: user.nome, email: user.email, papel:
user.papel },
    });
  } catch (erro) {
    console.error(erro);
    return res.status(500).json({ message: 'Erro interno ao autenticar' });
  }
}

// POST /api/auth/logout
export function logout(req, res) {
  // clearCookie precisa das mesmas opções usadas ao criar o cookie
  res.clearCookie('token', baseCookie);
  return res.status(200).json({ message: 'Logout realizado' });
}

// GET /api/auth/me (rota protegida): devolve o usuário logado; o front usa
para saber se há sessão
export async function me(req, res) {
  try {
    // req.userId foi colocado pelo middleware protegerRota
    const user = await User.findById(req.userId);
    if (!user) {
      return res.status(404).json({ message: 'Usuário não encontrado' });
    }
    return res.status(200).json({ id: user._id, nome: user.nome, email:
user.email, papel: user.papel });
  } catch (erro) {
    console.error(erro);
    return res.status(500).json({ message: 'Erro interno' });
  }
}
```

3.7 Controller de tarefas (`src/controllers/taskController.js`)

```
// mongoose é importado para validar ids e montar ObjectId na agregação
import mongoose from 'mongoose';
// model de tarefas
import Task from '../models/Task.js';

// campos que o cliente PODE enviar; owner fica de fora para ninguém trocar o
dono (mass assignment)
const CAMPOS_PERMITIDOS = ['titulo', 'descricao', 'status', 'prioridade',
'prazo'];

// converte erros do Mongoose em resposta HTTP padronizada
function tratarErro(erro, res) {
  // falha de validação do schema
  if (erro.name === 'ValidationError') {
    const mensagens = Object.values(erro.errors).map((e) => e.message);
    return res.status(400).json({ message: mensagens.join(', ') });
  }
  // id em formato inválido
  if (erro.name === 'CastError') {
    return res.status(400).json({ message: 'Valor inválido para o campo ' +
erro.path });
  }
  // qualquer outro erro é falha do servidor
  console.error(erro);
  return res.status(500).json({ message: 'Erro interno do servidor' });
}

// GET /api/tasks?status=pendente : lista as tarefas do usuário logado
export async function listar(req, res) {
  try {
    // filtro-base: só tarefas do dono
    const filtro = { owner: req.userId };
    // filtro opcional pela query string
    if (req.query.status) {
      filtro.status = req.query.status;
    }
    // mais recentes primeiro
    const tarefas = await Task.find(filtro).sort({ createdAt: -1 });
    return res.status(200).json(tarefas);
  } catch (erro) {
    return tratarErro(erro, res);
  }
}
```

```
// GET /api/tasks/:id : detalhe de uma tarefa
export async function buscarPorId(req, res) {
  try {
    const { id } = req.params;
    // valida o formato do id antes de consultar
    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({ message: 'ID inválido' });
    }
    // filtra também por owner: o usuário só enxerga as próprias tarefas
    const tarefa = await Task.findOne({ _id: id, owner: req.userId });
    if (!tarefa) {
      return res.status(404).json({ message: 'Tarefa não encontrada' });
    }
    return res.status(200).json(tarefa);
  } catch (erro) {
    return tratarErro(erro, res);
  }
}

// POST /api/tasks : cria uma tarefa
export async function criar(req, res) {
  try {
    const { titulo, descricao, status, prioridade, prazo } = req.body;
    if (!titulo) {
      return res.status(400).json({ message: 'O título é obrigatório' });
    }
    // o owner vem do TOKEN (req.userId), nunca do body
    const tarefa = await Task.create({ titulo, descricao, status, prioridade,
prazo, owner: req.userId });
    // avisa todos os clientes conectados via Socket.io (Seção 5.1)
    req.app.get('io').emit('task:created', tarefa);
    return res.status(201).json(tarefa);
  } catch (erro) {
    return tratarErro(erro, res);
  }
}

// PUT /api/tasks/:id : atualiza uma tarefa
export async function atualizar(req, res) {
  try {
    const { id } = req.params;
    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({ message: 'ID inválido' });
    }
    // monta o objeto de atualização só com campos permitidos e realmente
enviados
```

```
    const campos = {};
    CAMPOS_PERMITIDOS.forEach((campo) => {
      if (req.body[campo] !== undefined) {
        campos[campo] = req.body[campo];
      }
    });
    // filtro com owner: se a tarefa for de outra pessoa, nada casa e cai no
404 (não revela que existe)
    // new: true devolve o documento atualizado; runValidators aplica enum,
min, max também no update
    const tarefa = await Task.findOneAndUpdate({ _id: id, owner: req.userId
}, campos, { new: true, runValidators: true });
    if (!tarefa) {
      return res.status(404).json({ message: 'Tarefa não encontrada' });
    }
    req.app.get('io').emit('task:updated', tarefa);
    return res.status(200).json(tarefa);
  } catch (erro) {
    return tratarErro(erro, res);
  }
}

// DELETE /api/tasks/:id : remove uma tarefa (aqui com 403 explícito para
demonstrar a diferença)
export async function remover(req, res) {
  try {
    const { id } = req.params;
    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({ message: 'ID inválido' });
    }
    // busca primeiro para diferenciar 404 (não existe) de 403 (existe, mas é
de outro usuário)
    const tarefa = await Task.findById(id);
    if (!tarefa) {
      return res.status(404).json({ message: 'Tarefa não encontrada' });
    }
    // toString() porque owner é ObjectId e req.userId é string
    if (tarefa.owner.toString() !== req.userId) {
      return res.status(403).json({ message: 'Você não pode remover tarefa de
outro usuário' });
    }
    // remove pelo _id
    await Task.findByIdAndDelete(id);
    req.app.get('io').emit('task:deleted', { id });
    return res.status(200).json({ message: 'Tarefa removida com sucesso' });
  } catch (erro) {
```

```
    return tratarErro(erro, res);
  }
}

// GET /api/tasks/stats : totais por status (alimenta o gráfico da Seção 5.2)
export async function estatisticas(req, res) {
  try {
    // aggregate NÃO converte string em ObjectId sozinho, por isso o new
ObjectId no $match
    const grupos = await Task.aggregate([
      { $match: { owner: new mongoose.Types.ObjectId(req.userId) } },
      { $group: { _id: '$status', total: { $sum: 1 } } },
    ]);
    // começa com zero em todos os status para o gráfico sempre ter as três
barras
    const resultado = { pendente: 0, andamento: 0, concluida: 0 };
    // preenche com os totais reais vindos do banco
    grupos.forEach((grupo) => {
      resultado[grupo._id] = grupo.total;
    });
    return res.status(200).json(resultado);
  } catch (erro) {
    return tratarErro(erro, res);
  }
}
```

3.8 Rotas

`src/routes/authRoutes.js`

```
// Router: mini-aplicação Express para agrupar rotas
import { Router } from 'express';
import { register, login, logout, me } from
'../controllers/authController.js';
import { protegerRota } from '../middlewares/authMiddleware.js';

const router = Router();

// rotas públicas
router.post('/register', register);
router.post('/login', login);
router.post('/logout', logout);
// rota protegida: o middleware roda ANTES do controller
router.get('/me', protegerRota, me);

// exporta para o server.js montar em /api/auth
export default router;
```

`src/routes/taskRoutes.js`

```
import { Router } from 'express';
import { listar, buscarPorId, criar, atualizar, remover, estatisticas } from
'../controllers/taskController.js';
import { protegerRota } from '../middlewares/authMiddleware.js';

const router = Router();

// aplica a proteção a TODAS as rotas deste arquivo, definidas abaixo
router.use(protegerRota);

// /stats precisa vir ANTES de /:id, senão o Express trata a palavra stats
como se fosse um id
router.get('/stats', estatisticas);
router.get('/', listar);
router.get('/:id', buscarPorId);
router.post('/', criar);
router.put('/:id', atualizar);
router.delete('/:id', remover);

export default router;
```

> `src/routes/reportRoutes.js` (PDF) e `src/sockets/socket.js` (tempo real) são entregues completos na **Seção 5**; o `server.js` já importa os dois.

3.9 Testando o backend pelo terminal

```
# 1) saúde da API
curl http://localhost:3000/api/health

# 2) cadastro (esperado: 201)
curl -i -X POST http://localhost:3000/api/auth/register -H 'Content-Type:
application/json' -d
'{"nome":"Ana","email":"ana@teste.com","senha":"123456"}'

# 3) login: -c cookies.txt SALVA o cookie recebido (esperado: 200 + Set-
Cookie)
curl -i -c cookies.txt -X POST http://localhost:3000/api/auth/login -H
'Content-Type: application/json' -d
'{"email":"ana@teste.com","senha":"123456"}'

# 4) cria tarefa: -b cookies.txt ENVIA o cookie salvo (esperado: 201)
curl -i -b cookies.txt -X POST http://localhost:3000/api/tasks -H 'Content-
Type: application/json' -d '{"titulo":"Estudar Mongoose"}'

# 5) lista tarefas (esperado: 200) e totais por status
curl -b cookies.txt http://localhost:3000/api/tasks
curl -b cookies.txt http://localhost:3000/api/tasks/stats

# 6) sem cookie (esperado: 401)
curl -i http://localhost:3000/api/tasks
```

Status	Quando usar neste projeto
200 OK	Leitura, atualização, login e logout bem-sucedidos
201 Created	Cadastro de usuário e criação de tarefa
400 Bad Request	Campo faltando, ValidationError, CastError, e-mail duplicado, id inválido
401 Unauthorized	Sem token, token inválido/expirado, credenciais erradas
403 Forbidden	Autenticado, mas sem permissão (tarefa de outro usuário, rota de admin)
404 Not Found	Documento ou rota inexistente
500 Internal Server Error	Falha inesperada capturada no `catch`

SEÇÃO 4: BOILERPLATE FRONTEND (VITE + REACT)

Ordem sugerida na prova: arquivos-base (4.0), `api.js` (4.2), `PrivateRoute` e `App.jsx` (4.1), `Auth.jsx` (4.3) e `Dashboard.jsx` (4.4).

4.0 Arquivos-base do projeto

`frontend/.env` (variáveis do Vite SÓ são expostas se começarem com `VITE_`; reinicie o `npm run dev` ao alterar):

```
VITE_API_URL=http://localhost:3000/api
VITE_SOCKET_URL=http://localhost:3000
```

`frontend/index.html`

```
<!doctype html>
<html lang='pt-BR'>
  <head>
    <meta charset='UTF-8' />
    <meta name='viewport' content='width=device-width, initial-scale=1.0' />
    <title>Prova Full-Stack</title>
  </head>
  <body>
    <!-- o React monta toda a aplicação dentro desta div -->
    <div id='root'></div>
    <!-- ponto de entrada do Vite: carrega o main.jsx como módulo ES -->
    <script type='module' src='/src/main.jsx'></script>
  </body>
</html>
```

`frontend/vite.config.js`

```
// função que tipa e valida a configuração do Vite
import { defineConfig } from 'vite';
// plugin que compila JSX e habilita o Fast Refresh
import react from '@vitejs/plugin-react';
// plugin oficial do Tailwind v4 (substitui PostCSS e tailwind.config.js)
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  // a ordem dos plugins não é crítica aqui
  plugins: [react(), tailwindcss()],
  // porta fixa do servidor de desenvolvimento (precisa bater com CLIENT_URL
do backend)
  server: { port: 5173 },
});
```

`frontend/src/index.css` (uma linha ativa o Tailwind v4 inteiro)

```
@import 'tailwindcss';
```

`frontend/src/main.jsx`

```
// StrictMode: detecta efeitos colaterais perigosos em desenvolvimento
import { StrictMode } from 'react';
// createRoot: API de renderização do React 18 e 19
import { createRoot } from 'react-dom/client';
// BrowserRouter habilita as rotas em TODA a árvore de componentes
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
// importa o CSS global com o Tailwind
import './index.css';

// encontra a div#root do index.html e renderiza a aplicação dentro dela
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

4.1 Configuração de rotas (`src/App.jsx` e `src/components/PrivateRoute.jsx`)

`src/App.jsx`

```
// Routes agrupa as rotas; Route define cada uma; Navigate redireciona por
declaração
import { Routes, Route, Navigate } from 'react-router-dom';
import Auth from './pages/Auth.jsx';
import Dashboard from './pages/Dashboard.jsx';
import PrivateRoute from './components/PrivateRoute.jsx';

export default function App() {
  return (
    <Routes>
      {/* ROTAS PÚBLICAS: key diferente força o React a criar uma instância
nova e limpar o formulário ao trocar de tela */}
      <Route path='/login' element={<Auth key='login' modo='login' />} />
      <Route path='/cadastro' element={<Auth key='cadastro' modo='cadastro'
/>} />

      {/* ROTAS PRIVADAS: a rota-pai sem path usa PrivateRoute como guarda;
as filhas aparecem no Outlet dele */}
      <Route element={<PrivateRoute />}>
        <Route path='/dashboard' element={<Dashboard />} />
      </Route>

      {/* a raiz manda para o dashboard (que redireciona ao login se não
houver sessão) */}
      <Route path='/' element={<Navigate to='/dashboard' replace />} />
      {/* qualquer rota desconhecida cai no login */}
      <Route path='*' element={<Navigate to='/login' replace />} />
    </Routes>
  );
}
```

`src/components/PrivateRoute.jsx` (guarda de rotas: pergunta ao backend se o cookie é válido)

```
import { useEffect, useState } from 'react';
// Outlet renderiza a rota filha; Navigate redireciona
import { Navigate, Outlet } from 'react-router-dom';
import api from '../services/api.js';

export default function PrivateRoute() {
  // três estados possíveis: carregando, autenticado, negado
  const [estado, setEstado] = useState('carregando');
  // dados do usuário logado, repassados às rotas filhas
  const [usuario, setUsuario] = useState(null);

  // roda UMA vez na montagem: consulta /auth/me (o cookie vai junto por
```

```
causa do withCredentials)
  useEffect(() => {
    // flag para não atualizar estado se o componente desmontar antes da
resposta
    let ativo = true;
    const verificar = async () => {
      try {
        const resposta = await api.get('/auth/me');
        if (ativo) {
          setUsuario(resposta.data);
          setEstado('autenticado');
        }
      } catch (erro) {
        // 401 do backend: sem sessão válida
        if (ativo) {
          setEstado('negado');
        }
      }
    };
    verificar();
    // cleanup: marca como inativo na desmontagem
    return () => {
      ativo = false;
    };
  }, []);

  // enquanto a API não responde, mostra aviso (evita piscar a tela de login)
  if (estado === 'carregando') {
    return <p className='p-8 text-gray-600'>Verificando sessão</p>;
  }
  // sem sessão: redireciona; replace impede voltar para a rota privada com o
botão voltar
  if (estado === 'negado') {
    return <Navigate to='/login' replace />;
  }
  // com sessão: renderiza a rota filha e entrega o usuário via context (lido
com useOutletContext)
  return <Outlet context={{ usuario }} />;
}
```

4.2 Cliente HTTP (`src/services/api.js`)

```
import axios from 'axios';

// instância base: toda chamada api.get('/x') vira GET
http://localhost:3000/api/x
```

```
const api = axios.create({
  // lê do .env do Vite; se faltar, usa o endereço local padrão
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000/api',
  // ESSENCIAL: faz o navegador enviar e receber cookies em requisições entre
origens diferentes
  withCredentials: true,
  // todo body é enviado como JSON
  headers: { 'Content-Type': 'application/json' },
  // aborta se a API não responder em 10 segundos
  timeout: 10000,
});

// extrai a mensagem de erro padronizada do backend (campo message), com
texto reserva
export function mensagemDeErro(erro) {
  return erro.response?.data?.message || 'Não foi possível conectar ao
servidor';
}

// exporta a instância para ser usada em todo o projeto
export default api;
```

4.3 Formulário de Cadastro/Login (`src/pages/Auth.jsx`)

Um único componente atende as duas telas, controlado pela prop `modo`.

```
import { useState } from 'react';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import api, { mensagemDeErro } from '../services/api.js';

export default function Auth({ modo }) {
  // true quando a rota é /cadastro
  const ehCadastro = modo === 'cadastro';
  // função para redirecionar após o sucesso
  const navigate = useNavigate();
  // location.state carrega a mensagem enviada pelo navigate do cadastro
  const location = useLocation();

  // um único estado guarda todos os campos do formulário
  const [form, setForm] = useState({ nome: '', email: '', senha: '' });
  // mensagem de erro exibida no alerta vermelho
  const [erro, setErro] = useState('');
  // trava o botão durante a requisição
  const [carregando, setCarregando] = useState(false);

  // atualiza o campo digitado: e.target.name casa com a chave do estado
```

```
(spread copia o objeto)
  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  // dispara no envio do formulário
  const handleSubmit = async (e) => {
    // impede o reload padrão do navegador ao enviar <form>
    e.preventDefault();
    setErro('');
    setCarregando(true);
    try {
      if (ehCadastro) {
        // cria a conta e manda para o login com mensagem de sucesso
        await api.post('/auth/register', form);
        navigate('/login', { state: { sucesso: 'Cadastro realizado! Faça
login.' } });
      } else {
        // o backend responde com Set-Cookie; o navegador guarda sozinho
        await api.post('/auth/login', { email: form.email, senha: form.senha
});
        navigate('/dashboard');
      }
    } catch (err) {
      // 400 e 401 chegam aqui com a mensagem do backend
      setErro(mensagemDeErro(err));
    } finally {
      // roda sempre, com sucesso ou erro
      setCarregando(false);
    }
  };

  return (
    <div className='flex min-h-screen items-center justify-center bg-gray-
100'>
      <form onSubmit={handleSubmit} className='w-full max-w-md space-y-4
rounded-xl bg-white p-8 shadow-md'>
        <h1 className='text-center text-2xl font-bold text-gray-800'>
{ehCadastro ? 'Criar conta' : 'Entrar'}</h1>

        {/* mensagem de sucesso vinda do cadastro */}
        {location.state?.sucesso && (
          <p className='rounded-lg border border-green-300 bg-green-50 px-4
py-2 text-sm text-green-700'>{location.state.sucesso}</p>
        )}
```

```
        {/* alerta de erro: só aparece quando há mensagem */}
        {erro && <p className='rounded-lg border border-red-300 bg-red-50 px-
4 py-2 text-sm text-red-700'>{erro}</p>}

        {/* campo nome só no cadastro */}
        {ehCadastro && (
          <input
            type='text'
            name='nome'
            value={form.nome}
            onChange={handleChange}
            placeholder='Nome'
            required
            className='w-full rounded-lg border border-gray-300 px-3 py-2
focus:border-blue-500 focus:ring-2 focus:ring-blue-500 focus:outline-hidden'
          />
        )}

        <input
          type='email'
          name='email'
          value={form.email}
          onChange={handleChange}
          placeholder='E-mail'
          required
          className='w-full rounded-lg border border-gray-300 px-3 py-2
focus:border-blue-500 focus:ring-2 focus:ring-blue-500 focus:outline-hidden'
        />

        <input
          type='password'
          name='senha'
          value={form.senha}
          onChange={handleChange}
          placeholder='Senha (mínimo 6 caracteres)'
          required
          className='w-full rounded-lg border border-gray-300 px-3 py-2
focus:border-blue-500 focus:ring-2 focus:ring-blue-500 focus:outline-hidden'
        />

        <button
          type='submit'
          disabled={carregando}
          className='w-full rounded-lg bg-blue-600 px-4 py-2 font-semibold
text-white transition hover:bg-blue-700 disabled:cursor-not-allowed
disabled:opacity-50'
```

```
        >
          {carregando ? 'Enviando' : ehCadastro ? 'Cadastrar' : 'Entrar'}
        </button>

        {/* link para alternar entre as duas telas */}
        <p className='text-center text-sm text-gray-600'>
          {ehCadastro ? 'Já tem conta? ' : 'Não tem conta? '}
          <Link to={ehCadastro ? '/login' : '/cadastro'} className='font-
semibold text-blue-600 hover:underline'>
            {ehCadastro ? 'Entrar' : 'Cadastrar'}
          </Link>
        </p>
      </form>
    </div>
  );
}
```

4.4 Dashboard protegida (`src/pages/Dashboard.jsx`)

```
import { useEffect, useState } from 'react';
// useOutletContext lê o usuário entregue pelo PrivateRoute
import { useNavigate, useOutletContext } from 'react-router-dom';
import api, { mensagemDeErro } from '../services/api.js';

export default function Dashboard() {
  // usuário logado (veio do /auth/me no PrivateRoute)
  const { usuario } = useOutletContext();
  const navigate = useNavigate();

  // lista de tarefas vinda da API
  const [tarefas, setTarefas] = useState([]);
  // campo controlado do formulário de nova tarefa
  const [titulo, setTitulo] = useState('');
  // true até a primeira resposta chegar
  const [carregando, setCarregando] = useState(true);
  // mensagem de erro global da tela
  const [erro, setErro] = useState('');

  // busca as tarefas protegidas (o cookie vai automaticamente)
  const carregarTarefas = async () => {
    try {
      setErro('');
      const resposta = await api.get('/tasks');
      setTarefas(resposta.data);
    } catch (err) {
      setErro(mensagemDeErro(err));
```

```
    } finally {
      setCarregando(false);
    }
  };

  // array vazio: carrega os dados UMA vez, logo que a tela monta
  useEffect(() => {
    carregarTarefas();
  }, []);

  // cria uma tarefa e a coloca no topo da lista sem precisar recarregar tudo
  const criarTarefa = async (e) => {
    e.preventDefault();
    if (!titulo.trim()) {
      return;
    }
    try {
      const resposta = await api.post('/tasks', { titulo });
      // forma funcional do setState: novo item + itens anteriores
      setTarefas((anteriores) => [resposta.data, ...anteriores]);
      setTitulo('');
    } catch (err) {
      setErro(mensagemDeErro(err));
    }
  };

  // alterna entre concluída e pendente via PUT
  const alternarStatus = async (tarefa) => {
    const novoStatus = tarefa.status === 'concluida' ? 'pendente' :
'concluida';
    try {
      const resposta = await api.put('/tasks/' + tarefa._id, { status:
novoStatus });
      // substitui só a tarefa alterada, mantendo as demais
      setTarefas((anteriores) => anteriores.map((t) => (t._id === tarefa._id
? resposta.data : t)));
    } catch (err) {
      setErro(mensagemDeErro(err));
    }
  };

  // remove no backend e depois tira da lista local
  const removerTarefa = async (id) => {
    try {
      await api.delete('/tasks/' + id);
      setTarefas((anteriores) => anteriores.filter((t) => t._id !== id));
```

```
    } catch (err) {
      setErro(mensagemDeErro(err));
    }
  };

  // encerra a sessão: o backend limpa o cookie; depois volta ao login
  const sair = async () => {
    try {
      await api.post('/auth/logout');
    } finally {
      navigate('/login', { replace: true });
    }
  };

  return (
    <div className='min-h-screen bg-gray-100'>
      {/* cabeçalho com nome do usuário e botão de sair */}
      <header className='flex items-center justify-between bg-white px-8 py-4
shadow-md'>
        <h1 className='text-xl font-bold text-gray-800'>Painel de
{usuario.nome}</h1>
        <button onClick={sair} className='rounded-lg bg-red-600 px-3 py-1
text-sm font-semibold text-white hover:bg-red-700'>
          Sair
        </button>
      </header>

      <main className='mx-auto max-w-3xl space-y-6 p-8'>
        {erro && <p className='rounded-lg border border-red-300 bg-red-50 px-
4 py-2 text-sm text-red-700'>{erro}</p>}

        {/* formulário de nova tarefa */}
        <form onSubmit={criarTarefa} className='flex gap-2'>
          <input
            value={titulo}
            onChange={(e) => setTitulo(e.target.value)}
            placeholder='Nova tarefa'
            className='flex-1 rounded-lg border border-gray-300 px-3 py-2
focus:border-blue-500 focus:ring-2 focus:ring-blue-500 focus:outline-hidden'
          />
          <button type='submit' className='rounded-lg bg-blue-600 px-4 py-2
font-semibold text-white hover:bg-blue-700'>
            Adicionar
          </button>
        </form>
```

```
        {/* renderização condicional: carregando, vazio ou lista */}
        {carregando ? (
          <p className='text-gray-600'>Carregando tarefas</p>
        ) : tarefas.length === 0 ? (
          <p className='text-gray-600'>Nenhuma tarefa cadastrada.</p>
        ) : (
          <ul className='space-y-2'>
            {/* key obrigatória e única em listas */}
            {tarefas.map((tarefa) => (
              <li key={tarefa._id} className='flex items-center justify-
between rounded-lg bg-white p-4 shadow-md'>
                <span className={tarefa.status === 'concluida' ? 'text-gray-
400 line-through' : 'text-gray-800'}>{tarefa.titulo}</span>
                <div className='flex gap-2'>
                  <button onClick={() => alternarStatus(tarefa)}
className='rounded-lg bg-green-600 px-3 py-1 text-sm font-semibold text-white
hover:bg-green-700'>
                    {tarefa.status === 'concluida' ? 'Reabrir' : 'Concluir'}
                  </button>
                  <button onClick={() => removerTarefa(tarefa._id)}
className='rounded-lg bg-red-600 px-3 py-1 text-sm font-semibold text-white
hover:bg-red-700'>
                    Excluir
                  </button>
                </div>
              </li>
            ))}
          </ul>
        )}
      </main>
    </div>
  );
}
```

4.5 Diagnóstico rápido de erros comuns

Sintoma	Causa provável	Correção
Console: *blocked by CORS policy*	`origin` do CORS diferente da URL do front	`CLIENT_URL` exato, sem barra final, e `credentials: true`
Login retorna 200 mas `/auth/me` dá 401	Cookie não é enviado	`withCredentials: true` no Axios e `credentials: true` no CORS

Sintoma	Causa provável	Correção
`Cannot POST /auth/login`	Faltou o prefixo `/api`	Conferir `baseURL` e `app.use('/api/auth', ...)`
`EADDRINUSE: port 3000`	Porta ocupada por outro processo	Trocar `PORT` no `.env` ou encerrar o processo antigo
`import.meta.env.VITE_API_URL` é `undefined`	Variável sem prefixo `VITE_` ou servidor não reiniciado	Renomear e reiniciar o `npm run dev`
Socket não conecta	`listen` feito no `app` e não no `httpServer`	Usar `httpServer.listen`
`MongooseServerSelectionError`	MongoDB desligado	Iniciar o serviço (seção 1.1)

SEÇÃO 5: MÓDULOS ESPECIAIS E SNIPPETS PRONTOS

5.1 Tempo real com Socket.io

Servidor (`backend/src/sockets/socket.js`)

```
// recebe a instância io criada no server.js e registra todos os eventos
export default function registrarSockets(io) {
  // dispara a cada novo cliente conectado; socket representa ESSE cliente específico
  io.on('connection', (socket) => {
    // socket.id é único por conexão
    console.log('Cliente conectado:', socket.id);

    // io.emit: envia para TODOS os conectados, incluindo o próprio cliente que acabou de entrar
    io.emit('usuarios:online', io.sockets.sockets.size);

    // evento customizado recebido do cliente: o nome ('chat:enviar') precisa ser idêntico nos dois lados
    socket.on('chat:enviar', (dados) => {
      // sanitiza e limita o tamanho dos textos recebidos (nunca confie no cliente)
      const mensagem = {
        autor: String((dados && dados.autor) || 'Anônimo').slice(0, 50),
        texto: String((dados && dados.texto) || '').slice(0, 300),
```

```
        hora: new Date().toISOString(),
      };
      // ignora mensagens vazias
      if (!mensagem.texto.trim()) {
        return;
      }
      // io.emit: todos recebem, inclusive quem enviou (o remetente vê a
própria mensagem)
      io.emit('chat:nova', mensagem);
    });

    // indicador de digitação
    socket.on('chat:digitando', (autor) => {
      // socket.broadcast.emit: todos RECEBEM, MENOS quem enviou (ninguém
precisa ver que ele mesmo está digitando)
      socket.broadcast.emit('chat:digitando', String(autor ||
'Alguém').slice(0, 50));
    });

    // salas: agrupam sockets para mensagens direcionadas
    socket.on('sala:entrar', (sala) => {
      // coloca este socket dentro da sala
      socket.join(String(sala));
      // io.to(sala).emit: envia só para quem está na sala
      io.to(String(sala)).emit('sala:aviso', socket.id + ' entrou na sala ' +
sala);
    });

    // dispara quando o cliente fecha a aba, perde a rede ou chama
socket.disconnect()
    socket.on('disconnect', (motivo) => {
      console.log('Cliente desconectado:', socket.id, 'motivo:', motivo);
      // atualiza o contador para os que sobraram (o socket já foi removido
da lista neste momento)
      io.emit('usuarios:online', io.sockets.sockets.size);
    });
  });
}
```

Chamada	Quem recebe
`socket.emit('x', dado)`	Somente o cliente dono deste socket
`socket.broadcast.emit('x', dado)`	Todos, **exceto** o dono do socket
`io.emit('x', dado)`	**Todos** os conectados, incluindo o dono

Chamada	Quem recebe
`io.to('sala').emit('x', dado)`	Todos dentro da sala
`req.app.get('io').emit(...)`	Mesmo que `io.emit`, usado dentro de controllers REST (3.7)

Cliente React (`frontend/src/components/TarefasTempoReal.jsx`)

```
import { useEffect, useRef, useState } from 'react';
// io é a função que abre a conexão com o servidor
import { io } from 'socket.io-client';

export default function TarefasTempoReal({ nomeUsuario }) {
  // useRef guarda o socket SEM provocar re-renderização e mantém a mesma
instância entre renders
  const socketRef = useRef(null);
  // guarda o id do timeout do indicador de digitação para poder cancelar
  const timerRef = useRef(null);

  const [conectado, setConectado] = useState(false);
  const [online, setOnline] = useState(0);
  // feed de eventos disparados pela API REST (criação, edição, remoção)
  const [eventos, setEventos] = useState([]);
  // mensagens do chat
  const [mensagens, setMensagens] = useState([]);
  const [texto, setTexto] = useState('');
  const [digitando, setDigitando] = useState('');

  // abre a conexão UMA vez na montagem e fecha na desmontagem
  useEffect(() => {
    // cria a conexão; withCredentials combina com credentials true do CORS
do servidor
    const socket = io(import.meta.env.VITE_SOCKET_URL ||
'http://localhost:3000', { withCredentials: true });
    socketRef.current = socket;

    // adiciona uma linha no feed, mantendo apenas as 10 mais recentes
    const registrar = (linha) => {
      setEventos((anteriores) => [linha, ...anteriores].slice(0, 10));
    };

    // eventos nativos de ciclo de vida da conexão
    socket.on('connect', () => setConectado(true));
    socket.on('disconnect', () => setConectado(false));
```

```
    // eventos customizados vindos do servidor: o primeiro argumento é o
nome, o segundo recebe o dado
    socket.on('usuarios:online', (total) => setOnline(total));
    socket.on('task:created', (tarefa) => registrar('Nova tarefa: ' +
tarefa.titulo));
    socket.on('task:updated', (tarefa) => registrar('Tarefa atualizada: ' +
tarefa.titulo + ' (' + tarefa.status + ')'));
    socket.on('task:deleted', (dados) => registrar('Tarefa removida: ' +
dados.id));
    socket.on('chat:nova', (mensagem) => setMensagens((anteriores) =>
[...anteriores, mensagem]));
    socket.on('chat:digitando', (autor) => {
      setDigitando(autor + ' está digitando');
      // reinicia o temporizador a cada nova digitação e apaga o aviso após
1,5 segundo
      clearTimeout(timerRef.current);
      timerRef.current = setTimeout(() => setDigitando(''), 1500);
    });

    // CLEANUP: sem isto cada montagem (inclusive a dupla do StrictMode)
deixaria uma conexão aberta
    return () => {
      clearTimeout(timerRef.current);
      socket.disconnect();
    };
  }, []);

  // emite o evento de chat para o servidor
  const enviar = (e) => {
    e.preventDefault();
    if (!texto.trim()) {
      return;
    }
    socketRef.current.emit('chat:enviar', { autor: nomeUsuario, texto });
    setTexto('');
  };

  // avisa o servidor a cada tecla; o servidor repassa aos OUTROS com
broadcast
  const aoDigitar = (e) => {
    setTexto(e.target.value);
    socketRef.current.emit('chat:digitando', nomeUsuario);
  };

  return (
    <section className='space-y-4 rounded-xl bg-white p-4 shadow-md'>
```

```
      <div className='flex items-center justify-between'>
        <h2 className='text-lg font-semibold text-gray-800'>Tempo real</h2>
        <span className={conectado ? 'text-sm font-semibold text-green-600' :
'text-sm font-semibold text-red-600'}>
          {conectado ? 'Conectado' : 'Desconectado'} · {online} online
        </span>
      </div>

      {/* feed de eventos da API */}
      <ul className='space-y-1 text-sm text-gray-700'>
        {eventos.length === 0 && <li className='text-gray-400'>Aguardando
eventos de tarefas</li>}
        {eventos.map((linha, indice) => (
          <li key={indice}>{linha}</li>
        ))}
      </ul>

      {/* chat */}
      <div className='h-40 space-y-1 overflow-y-auto rounded-lg border
border-gray-300 p-2 text-sm'>
        {mensagens.map((m, indice) => (
          <p key={indice}>
            <strong>{m.autor}:</strong> {m.texto}
          </p>
        ))}
      </div>
      <p className='h-4 text-xs text-gray-500'>{digitando}</p>
      <form onSubmit={enviar} className='flex gap-2'>
        <input
          value={texto}
          onChange={aoDigitar}
          placeholder='Mensagem'
          className='flex-1 rounded-lg border border-gray-300 px-3 py-2
focus:border-blue-500 focus:ring-2 focus:ring-blue-500 focus:outline-hidden'
        />
        <button type='submit' className='rounded-lg bg-blue-600 px-4 py-2
font-semibold text-white hover:bg-blue-700'>
          Enviar
        </button>
      </form>
    </section>
  );
}
```

5.2 Gráficos com ApexCharts (`frontend/src/components/GraficoTarefas.jsx`)

```
import { useEffect, useState } from 'react';
// componente do wrapper react-apexcharts (o pacote apexcharts precisa estar instalado junto)
import Chart from 'react-apexcharts';
import api, { mensagemDeErro } from '../services/api.js';

// versao é uma string que muda quando as tarefas mudam; ela dispara nova busca dos totais
export default function GraficoTarefas({ versao = '' }) {
  // dados do gráfico guardados em estado: quando mudam, o gráfico se redesenha sozinho
  const [totais, setTotais] = useState({ pendente: 0, andamento: 0, concluida: 0 });
  // tipo do gráfico escolhido pelo usuário: bar ou line
  const [tipo, setTipo] = useState('bar');
  const [erro, setErro] = useState('');

  // refaz a busca sempre que a prop versao mudar (array de dependências com [versao])
  useEffect(() => {
    const carregar = async () => {
      try {
        const resposta = await api.get('/tasks/stats');
        setTotais(resposta.data);
        setErro('');
      } catch (err) {
        setErro(mensagemDeErro(err));
      }
    };
    carregar();
  }, [versao]);

  // options: aparência e eixos do gráfico
  const options = {
    // id identifica o gráfico; toolbar escondida deixa o visual limpo
    chart: { id: 'tarefas-por-status', toolbar: { show: false } },
    // rótulos do eixo X na MESMA ordem dos valores da série
    xaxis: { categories: ['Pendente', 'Em andamento', 'Concluída'] },
    // paleta de cores
    colors: ['#2563eb'],
    // mostra o número em cima de cada barra ou ponto
    dataLabels: { enabled: true },
    // curva suave e espessura só quando o tipo é linha
    stroke: { curve: 'smooth', width: tipo === 'line' ? 3 : 0 },
```

```
    // título do gráfico
    title: { text: 'Tarefas por status', align: 'left' },
    // cantos arredondados e largura das colunas (só vale para barras)
    plotOptions: { bar: { borderRadius: 4, columnWidth: '45%' } },
  };

  // series: lista de séries; cada uma tem nome e array de valores (mesma
ordem das categorias)
  const series = [{ name: 'Tarefas', data: [totais.pendente,
totais.andamento, totais.concluida] }];

  return (
    <section className='rounded-xl bg-white p-4 shadow-md'>
      {/* seletor do tipo de gráfico: trocar o estado troca o gráfico */}
      <select
        value={tipo}
        onChange={(e) => setTipo(e.target.value)}
        className='mb-2 rounded-lg border border-gray-300 px-2 py-1 text-sm'
      >
        <option value='bar'>Barras</option>
        <option value='line'>Linha</option>
      </select>

      {erro && <p className='mb-2 rounded-lg border border-red-300 bg-red-50
px-4 py-2 text-sm text-red-700'>{erro}</p>}

      {/* key={tipo} força recriar o gráfico ao trocar o tipo (evita resíduos
visuais) */}
      <Chart key={tipo} options={options} series={series} type={tipo} height=
{300} />
    </section>
  );
}
```

Variação rosca ou pizza: em `donut` e `pie` a `series` é um array simples de números e os rótulos vão em `labels`:

```
// type='donut' (ou 'pie'): series = [3, 2, 5] e options = { labels:
['Pendente', 'Em andamento', 'Concluída'] }
<Chart type='donut' series={[totais.pendente, totais.andamento,
totais.concluida]} options={{ labels: ['Pendente', 'Em andamento',
'Concluída'] }} height={300} />
```

5.3 Relatório em PDF com PDFKit

Backend (`backend/src/routes/reportRoutes.js`)

```
import { Router } from 'express';
// PDFDocument cria o documento e funciona como um stream de escrita
import PDFDocument from 'pdfkit';
import Task from '../models/Task.js';
import User from '../models/User.js';
import { protegerRota } from '../middlewares/authMiddleware.js';

const router = Router();

// traduz o valor do banco para texto de exibição
const ROTULOS = { pendente: 'Pendente', andamento: 'Em andamento', concluida:
'Concluída' };

// GET /api/reports/tasks : gera e envia o PDF das tarefas do usuário logado
router.get('/tasks', protegerRota, async (req, res) => {
  try {
    // busca os dados ANTES de começar a escrever o PDF (se falhar, ainda dá
para responder 500)
    const usuario = await User.findById(req.userId);
    const tarefas = await Task.find({ owner: req.userId }).sort({ createdAt:
-1 });

    // cria o documento A4 com margem de 40 pontos
    const doc = new PDFDocument({ size: 'A4', margin: 40 });

    // cabeçalhos HTTP: tipo do conteúdo e nome do arquivo; attachment força
o download
    res.setHeader('Content-Type', 'application/pdf');
    res.setHeader('Content-Disposition', 'attachment; filename=relatorio-
tarefas.pdf');

    // conecta a saída do PDF diretamente na resposta HTTP (stream, sem
salvar arquivo em disco)
    doc.pipe(res);

    // ---------- TÍTULO ----------
    doc.font('Helvetica-Bold').fontSize(20).text('Relatório de Tarefas', {
align: 'center' });
    // moveDown avança o cursor em linhas; 0.5 = meia linha
    doc.moveDown(0.5);
    doc.font('Helvetica').fontSize(10).text('Usuário: ' + usuario.nome + '  |
Gerado em: ' + new Date().toLocaleString('pt-BR'), { align: 'center' });
```

```
    doc.moveDown();

    // linha horizontal: moveTo(x, y) posiciona, lineTo desenha o traço,
stroke aplica
    doc.moveTo(40, doc.y).lineTo(555, doc.y).strokeColor('#000000').stroke();
    doc.moveDown();

    // ---------- TABELA ----------
    // posições X de cada coluna (A4 tem 595 de largura; área útil vai de 40
a 555)
    const colTitulo = 40;
    const colStatus = 300;
    const colPrioridade = 400;
    const colData = 470;

    // y guarda a posição vertical atual (controle manual das linhas)
    let y = doc.y;

    // faixa cinza do cabeçalho: rect(x, y, largura, altura).fill(cor)
    doc.rect(40, y - 4, 515, 20).fill('#e5e7eb');
    // volta a cor do texto para preto (fill muda a cor atual)
    doc.fillColor('#000000').font('Helvetica-Bold').fontSize(10);
    doc.text('Título', colTitulo, y, { lineBreak: false });
    doc.text('Status', colStatus, y, { lineBreak: false });
    doc.text('Prioridade', colPrioridade, y, { lineBreak: false });
    doc.text('Criada em', colData, y, { lineBreak: false });
    y += 22;

    // linhas de dados
    doc.font('Helvetica').fontSize(10);
    tarefas.forEach((tarefa) => {
      // controle de paginação: se passou de 760, abre página nova e reinicia
o y
      if (y > 760) {
        doc.addPage();
        y = 40;
      }
      // corta títulos muito longos para não invadir a coluna ao lado
      doc.text(tarefa.titulo.slice(0, 45), colTitulo, y, { lineBreak: false
});
      doc.text(ROTULOS[tarefa.status] || tarefa.status, colStatus, y, {
lineBreak: false });
      doc.text(String(tarefa.prioridade), colPrioridade, y, { lineBreak:
false });
      doc.text(new Date(tarefa.createdAt).toLocaleDateString('pt-BR'),
colData, y, { lineBreak: false });
```

```
      // linha fina separando as linhas da tabela
      doc.moveTo(40, y + 14).lineTo(555, y +
14).strokeColor('#d1d5db').stroke();
      y += 20;
    });

    // ---------- RODAPÉ ----------
    doc.font('Helvetica-Bold').fontSize(11).fillColor('#000000');
    doc.text('Total de tarefas: ' + tarefas.length, 40, y + 10, { lineBreak:
false });

    // finaliza o documento: fecha o stream e conclui a resposta HTTP
(obrigatório)
    doc.end();
  } catch (erro) {
    console.error(erro);
    // se nada foi enviado ainda, responde JSON de erro; se já começou o
stream, apenas encerra
    if (!res.headersSent) {
      return res.status(500).json({ message: 'Erro ao gerar o relatório' });
    }
    res.end();
  }
});

export default router;
```

Frontend (`frontend/src/components/BotaoRelatorio.jsx`)

```
import { useState } from 'react';
import api from '../services/api.js';

export default function BotaoRelatorio() {
  const [gerando, setGerando] = useState(false);
  const [erro, setErro] = useState('');

  const baixar = async () => {
    setGerando(true);
    setErro('');
    try {
      // responseType blob: o Axios entrega os bytes brutos do PDF em vez de
tentar ler como texto
      const resposta = await api.get('/reports/tasks', { responseType: 'blob'
});
      // cria uma URL temporária na memória do navegador apontando para o PDF
      const url = window.URL.createObjectURL(new Blob([resposta.data], {
```

```
type: 'application/pdf' }));
      // cria um link invisível, aciona o clique para baixar e remove em
seguida
      const link = document.createElement('a');
      link.href = url;
      link.download = 'relatorio-tarefas.pdf';
      document.body.appendChild(link);
      link.click();
      link.remove();
      // libera a memória da URL temporária
      window.URL.revokeObjectURL(url);
    } catch (err) {
      // em erro, o corpo também chega como blob; por isso a mensagem é fixa
      setErro('Não foi possível gerar o relatório');
    } finally {
      setGerando(false);
    }
  };

  return (
    <div className='space-y-2'>
      <button
        onClick={baixar}
        disabled={gerando}
        className='rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white
transition hover:bg-blue-700 disabled:cursor-not-allowed disabled:opacity-50'
      >
        {gerando ? 'Gerando PDF' : 'Baixar relatório em PDF'}
      </button>
      {erro && <p className='rounded-lg border border-red-300 bg-red-50 px-4
py-2 text-sm text-red-700'>{erro}</p>}
    </div>
  );
}
```

> Atalho sem Axios: `window.open('http://localhost:3000/api/reports/tasks')` também baixa o PDF, pois o navegador envia o cookie (mesmo site, portas diferentes). O método com Blob é o mais robusto e permite exibir erros.

5.4 Plugando os módulos no Dashboard

Em `src/pages/Dashboard.jsx`, **adicione estas três linhas de import** junto aos imports existentes:

```
import GraficoTarefas from '../components/GraficoTarefas.jsx';
import TarefasTempoReal from '../components/TarefasTempoReal.jsx';
import BotaoRelatorio from '../components/BotaoRelatorio.jsx';
```

E **insira este bloco** dentro do `<main>`, logo abaixo da linha `{erro &&` e acima do `<form onSubmit={criarTarefa}>`:

```
{/* versao muda quando qualquer tarefa é criada, removida ou tem o status
alterado, e o gráfico recarrega */}
<GraficoTarefas versao={tarefas.map((t) => t.status).join(',')} />
<BotaoRelatorio />
<TarefasTempoReal nomeUsuario={usuario.nome} />
```

5.5 Plano de ataque para a prova

1. Rodar o script da seção 1.5 e confirmar `npm run dev` nos dois projetos.
2. Criar `.env` do backend e `db.js`; subir o servidor e ver *MongoDB conectado*.
3. Models, middleware, controllers e rotas; testar com os `curl` da seção 3.9.
4. Criar `socket.js` e `reportRoutes.js` (o `server.js` importa os dois; sem eles o servidor não sobe).
5. Abrir `/api-docs` e conferir o Swagger.
6. Frontend: arquivos-base, `api.js`, `PrivateRoute`, `App.jsx`, `Auth.jsx`, `Dashboard.jsx`.
7. Testar cadastro, login, criação de tarefa, logout e acesso direto a `/dashboard` sem sessão.
8. Plugar socket, gráfico e PDF (seção 5.4) e abrir duas abas para ver o tempo real.
9. Em caso de erro, consultar a tabela da seção 4.5 antes de qualquer outra coisa.